



CD-Unit1-Notes - Compiler design unit 1 notes jntuh R18 regulation

Compiler Design (Jawaharlal Nehru Technological University, Hyderabad)



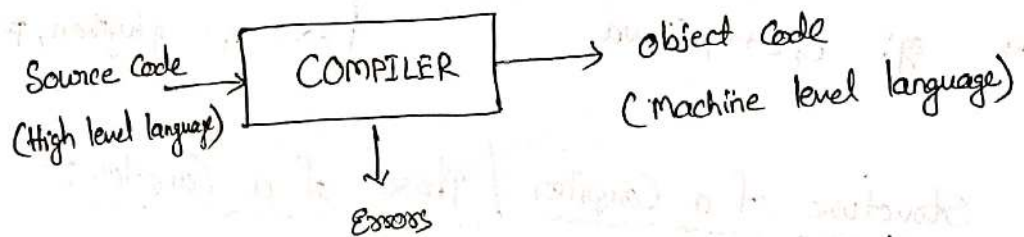
Scan to open on Studocu

Language Processors: A special translator system software used to translate the program written in high-level language into machine code is called Language Processors.

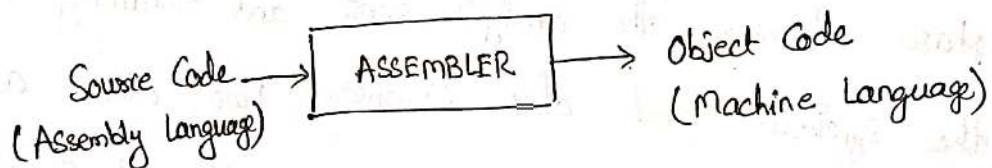
The language processor can be any of the following three types:-

(i) Compiler:- The language processor that reads the complete source program written in high level language as a whole in one go and translates it into an equivalent program into machine language is known as Compiler.

Ex:- C, C++, C#, Java.



(ii) Assembler:- The Assembler is used to translate the program written in Assembly language into machine code.



(iii) Interpreter:- The translation of single statement of source program into machine code is done by language processor and executes it before moving onto the next line is called an interpreter.

eg:- Perl, Python & Matlab.

Difference between Compiler & Interpreter :-

COMPILER

- Converts the entire source code into executable machine code.
- Compiler takes large amount of time to analyze the entire source code but the overall execution time of the program is comparatively faster.
- Generates intermediate object code.
- eg: C, C++, Java

INTERPRETER

- takes a source program & converts it into machine code line by line as it comes to it.
- Interpreter takes less amount of time to analyze the source code but the overall execution time of the program is slower.
- No intermediate object code is generated.
- eg: python, perl.

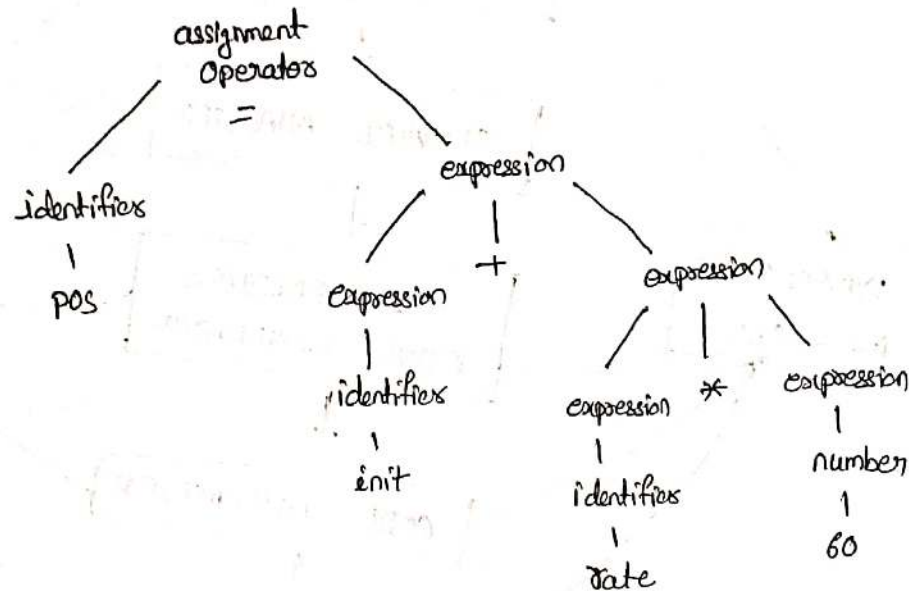
Structure of a Compiler / Phases of a Compiler :-

A compiler operates in 6 phases, each of which transfer source program from one representation to another. The first three phases forming the analysis part and remaining three phases forming the synthesis part of a compiler. Two other activities symbol table and error handler interacts with all 6 phases of compiler.

Symbol Table Management :- A symbol table is a data structure containing record for each identifier with fields for the attributes of the identifier. It allows us to find record of each identifier quickly & used to store and retrieve data from that record.

The blanks or white spaces separating the characters are removed during lexical analysis.

- ⑤ Hierarchical / Syntax Analysis / Parsing :- It involves grouping of the tokens of the source program into grammatical phrases that are used by compiler to synthesize output.



- ⑥ Semantic Analysis : The Semantic phase checks the source program for semantic errors and gather information for the subsequent code generation phase. It identifies the operators, operations of expressions & statements. The important component of semantic analysis is "typechecking".

- ⑦ Intermediate Code Generation :- After syntax & semantic analysis, some compilers generate an explicit intermediate representation of source program. The intermediate code has two important properties (i) easy to produce & (ii) easy to translate to the target program. A form of intermediate code is 3-address code which is like assembly language.

```
temp1 = intercal(60);
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

②
e) Code Optimization :- The code optimization phase attempts to improve the intermediate code.

eg: $templ = id3 * 60.0$
 $id1 = id2 + templ.$

② Code Generation: The final phase of the compiler is the generation of target code, consisting normally of relocatable machine code or assembly code. Memory locations are selected for each variable used by the program.

eg: $MOV\ F\ id3, R2$
 $MUL\ F\ \#60.0, R2$
 $MOV\ F\ id2, R1$
 $ADDF\ R2, R1$
 $MOV\ F\ R1, id1$

Difference between a pass & phase

A sub-process is called a phase. (or) A phase is a logical cohesive operation that takes one representation as input of source program & produces as output another representation.

The combination of one or two phases in a module is called a Pass. A Pass reads the source program or output of the previous pass which is read by subsequent pass.

In logical organization of a compiler, the front-end phases of compiler lexical analysis, syntax analysis, semantic analysis and intermediate code generation might be grouped together into one pass. Code optimization is an optional pass. Back-end pass consists of code generation for particular target machine.

Translation of a statement:

$pos = int1 + rate * 60$

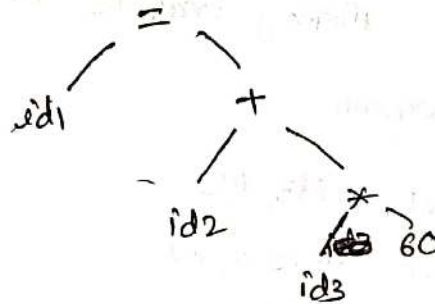


Lexical Analyzer

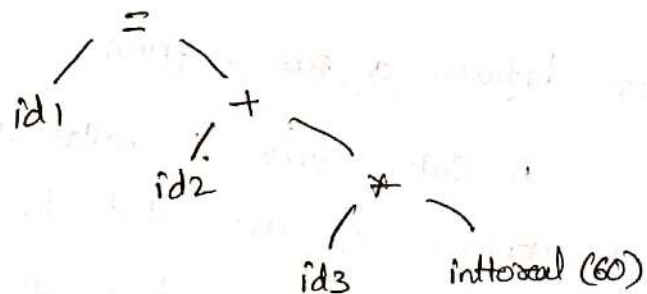
$id1 = id2 + id3 * 60$



Syntax Analyzer



Semantic Analyzer



Intermediate Code Generator

$temp1 = intoreal(60)$
 $temp2 = id3 * temp1$
 $temp3 = id2 + temp2$
 $id1 = temp3$



Code Optimizer

$temp1 = id3 * 60.0$
 $id1 = id2 + temp1$



Code Generator

MOVF $id3, R2$
MULF $\#60.0, R2$
MOVF $id2, R1$

ADDF $R2, R1$
MOVF $R1, id1$

Compiler Construction Tools :-

(4)

Tools use specified languages for specifying & implementing specific components. Some Compiler Construction tools are:

- (i) Parsers Generators :- produce syntax analyzer from grammatical description of a language.
- (ii) Scanner Generators :- produce lexical analyzer from a regular-expression description of a language.
- (iii) Syntax-Directed Translation Engines :- produce collection of routines for walking a parse tree & generating intermediate code.
- (iv) Code-generator Generators :- produce a code-generator from a collection of rules translating each operation of the intermediate language to machine language.
- (v) Data-Flow analysis Engines :- facilitate the gathering of information about how values are transmitted from one part of a program to other part.
- (vi) Compiler-Construction toolkits :- provide an integrated set of routines for constructing various phases of compiler.

The Science of Building a Compiler :-

Compiler design is a beautiful example where complicated, real world problems are solved by abstracting the essence of the problem mathematically.

Modeling in Compiler Design & Implementation :-

- Design right mathematical models & choose right algorithms.
- Some fundamental models are finite-state machine & regular expressions to describe lexical units of the program such as keywords, identifiers & so on.

The science of code optimization :-

Optimization in a compiler design refers to the attempts that a compiler makes to produce the code that is more efficient than the obvious code.

Compiler optimization must meet the following design objectives:

→ Optimization must be correct i.e. preserve the meaning of compiler program.

→ Optimization must improve performance of many programs.

→ Compilation time must be kept reasonable.

→ Engineering efforts required must be manageable.

Programming Language Basics :-

The most important terminology & distinctions in programming languages are

(i) Static / Dynamic distinction :

Static - the issue decided at compile-time.

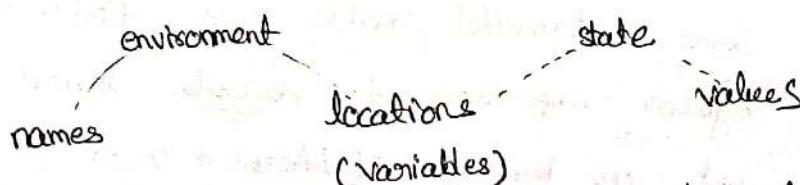
Dynamic - the issue decided at run-time.

Static scope / lexical scope - possible to determine the scope of a declaration by looking at the program.

Dynamic scope - as program runs refers to several different declarations.

(ii) Environment and States :

Environment is a mapping from names to locations in a store. (l-values). Environment changes according to scope rules of a language. State is a mapping from locations in store to their values.



Two-stage mapping from names to values

⑤
An identifier is a string of characters (letters or digits) that refer to an entity such as data object, procedure, class or type. All identifiers are names but all names are not identifiers.

- A variable/name refers to a particular location of the store.

(iii) Static Scope and Block Structure:

Most of the languages use static scope. The scope of declaration is determined implicitly where the declaration appears in the program. Languages such as C++, JAVA, C# provide explicit control over scopes through use of keywords like public, private and protected.

A block is grouping of declarations and statement. In C language a block use '{ & }' as delimiters.

(iv) Explicit Access Control:

Classes and structures introduce a new scope for their members. If P is an object of a class with a field x, then P.x refers to field x in class definition.

The use of keywords like public, private & protected in object-oriented languages provide explicit access to member names in a super class.

(v) Dynamic Scope:

Any scoping policy is dynamic if it is based on the factors that can be known only when program executes.

(vi) Parameter Passing Mechanisms :-

Call-by-value : The actual parameter is evaluated or copied. The value is placed in location belonging to the corresponding formal parameter of the called procedure. Call-by-value has the effect that all computation involving the formal parameters done by the called procedure is local to that procedure & actual parameter themselves cannot be changed.

Call-by-reference : - The address of the actual parameter is passed to the callee as the value of corresponding formal parameters. Changes to formal parameters thus appear as changes to the actual parameters.

(vii) Aliasing :

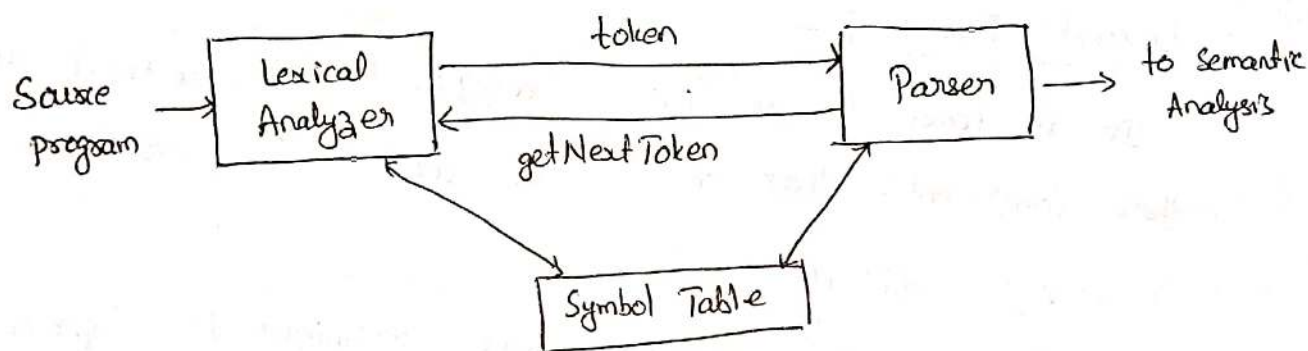
If two formal parameters refer to same ~~same~~ location, such variables are said to be aliases of one another.

eg! Suppose x is an array of procedure P & P calls another procedure $q(x, y)$ with a call $q(a, a)$. Since array names are references x & y have become aliases of each other.

LEXICAL ANALYSIS :-

6

Role of the Lexical Analyzer:- It is the first phase of the compiler. The main task of the lexical analyzer is to read the input characters of source program, group them into lexemes and produce as output a sequence of tokens. When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table. The getNextToken command, causes the lexical analyzer to read characters from its input until it can identify next lexeme & produce the next token, which it returns to the parser.



Lexical Analyzers are divided into ~~to~~ a cascade of two processes:

- Scanning consists of simple processes that do not require tokenization of the input such as deletion of comments, white spaces.
- Lexical Analysis is more complex portion, which produces tokens from the output of the scanner.

Lexical Analysis Versus Parsing :

The reasons for separating lexical analysis and syntax analysis are

- Simplicity of design.
- Compiler efficiency is improved.
- Compiler portability is enhanced.

Tokens, Pattern & Lexemes:

- A token is a pair consisting of a token name and an optional attribute value.
- A pattern is a description of the form that the lexemes of a token may take.
- A lexeme is a sequence of characters in the source program that matches the pattern for a token.

eg: `printf("Total = %.d\n", score);`

Both `printf` & `score` are lexemes matching the pattern for token identifier and `"Total = %.d\n"` is lexeme matching the literal.

Lexical Errors :-

It is hard for a lexical analyzer to tell, without the aid of other components that there is a source-code error.

eg: `fi(a == 10)`

The simplest error recovery technique is "panic mode" recovery. In this strategy successive characters are deleted until lexical analyzer finds a well-formed token.

Other possible error-recovery actions are:

- Delete one character from the remaining input
- Insert a missing character into remaining input.
- Replace a character by another character.
- Transpose two adjacent characters

Input Buffering:-

① Buffer Pairs: Buffering techniques are developed to reduce the amount of overhead required to process a single input character. In a Buffer pair two pointers to the input are maintained:

① Pointer lexemeBegin, marks the beginning of current lexeme whose extent we are attempting to determine.

② Pointer forward scans ahead until a pattern match is found.

② Sentinels: In Buffer pair, for each character we make two tests: one is end of the buffer and one to determine whether character is read. we can combine the buffer-end test with test for current character to hold a sentinel character at the end of buffer.

A sentinel is a special character that cannot be part of source program and a natural choice is the character ϵ .

Specification of Tokens:-

For specifying patterns to recognize tokens, Regular Expressions plays an important role. Regular expression will serve as names for set of strings. Strings are sequence of ~~characters~~ symbols drawn over some alphabets.

Strings & Languages:-

An alphabet is any finite set of symbols. Typical examples of symbols are letters, digits & punctuations.

Binary alphabet = $\{0, 1\}$

A string over an alphabet is a finite sequence of symbols. Sentence & word are often used as synonyms for string. The length of a string is denoted as $|S|$. Empty string is denoted by ϵ is of length zero.

A language is any countable set of strings over some fixed alphabet.

Terms for parts of a string: prefix, suffix, substring, proper prefixes & suffixes, subsequence of S .

Operations on Languages: Union, Concatenation, closure $\begin{cases} \rightarrow \text{Kleene} \\ \rightarrow \text{positive} \end{cases}$

Union: $L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$

Concatenation: $LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$

Kleene closure: $L^* = \bigcup_{i=0}^{\infty} L^i$

positive closure: $L^+ = \bigcup_{i=1}^{\infty} L^i$

Regular Expression: - Regular expressions are useful to represent some strings in algebraic form & these languages are accepted by Finite Automata.

(or)
Regular expression is a sequence of characters that specify the search pattern.

These are two basic rules that form basis for R.E (8)

- (i) ϵ is a regular expression and $L(\epsilon)$ is $\{\epsilon\}$ is the language whose sole member is empty string.
- (ii) If a is a symbol in Σ , then a is regular expression and $L(a) = \{a\}$ that is the language with one string of length one.

There are four parts to the induction where by larger regular expressions are built from smaller ones.

- (1) $(r)(s)$ is a regular expression denoting the language $L(r)L(s)$
- (2) $(r)(s)$ is a regular expression denoting the language $L(r) \cdot L(s)$.
- (3) $(r)^*$ is a regular expression denoting $(L(r))^*$
- (4) r is a regular expression denoting $L(r)$.

Regular expression for identifiers :

$id \rightarrow \text{letter_} (\text{letter_}/\text{digit})^*$

$\text{letter_} \rightarrow a|b|c \dots |z|A|B| \dots |Z| -$

$\text{digit} \rightarrow 0|1|2| \dots |9$

Regular expression for unsigned numbers : eg: 528, 32.7, 6.4E4

$\text{number} \rightarrow \text{digits optional_fraction optional_exponent}$

$\text{digits} \rightarrow \text{digit digit}^*$

$\text{optional_fraction} \rightarrow \text{digits} / \epsilon$

$\text{optional_exponent} \rightarrow (\epsilon (+|-)\text{digits}) / \epsilon$

$\text{digit} \rightarrow 0|1|2| \dots |9$

Regular expression for relational operators

"<" | "<=" | ">" | ">=" | "=" | "<>"

Extensions of Regular Expressions (Notational shortcuts):

(i) One or more instances: Unary operators +

$$s^+ \Rightarrow s.s^*$$

(ii) Zero or one instance: Unary postfix operators?

$$(s)? \rightarrow s/\epsilon$$

(iii) character classes: The notation $[abc]$ where a, b & c are alphabet symbols denote regular expression $a|b|c$.

* The main drawback of R.E is it cannot describe repeating strings.

Recognition of Tokens:-

stmt $\rightarrow$ If expr then stmt
 | If expr then stmt else stmt
 | ϵ

expr $\rightarrow$ term relop term
 | term

term $\rightarrow$ id
 | number

Tokens generated are

if $\rightarrow$ if

then $\rightarrow$ then

else $\rightarrow$ else

relop $\rightarrow$ $< | <= | > | >= | = | <>$

id $\rightarrow$ letter (letter/digit)⁺

number $\rightarrow$ digit⁺ (.digit⁺)? (E(+|-)?digit⁺)?

letter $\rightarrow$ a|b|c|d|...|z|A|B|C|...|Z|_

digit $\rightarrow$ 0|1|2|...|9

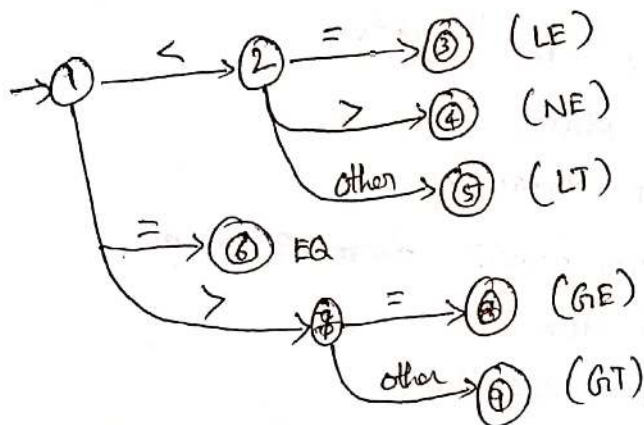
Lexical Analyzer will strip out white spaces as below (9)

delim $\rightarrow$ blank / tab / newline

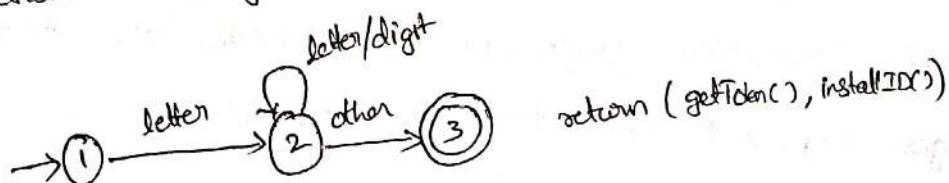
ws $\rightarrow$ delim⁺

Transition Diagram:- It depicts the action that takes place when a lexical analyzer is called by the parser to get next token. This consist of start state, end state, intermediate states. Initial state (start state) is represented by arrow into a circle ($\rightarrow \textcircled{1}$), end state is represented by concentric circles ($\textcircled{\textcircled{4}}$), intermediate state is represented by circles ($\textcircled{}$). Each intermediate stage is connected by arrows ($\rightarrow$).

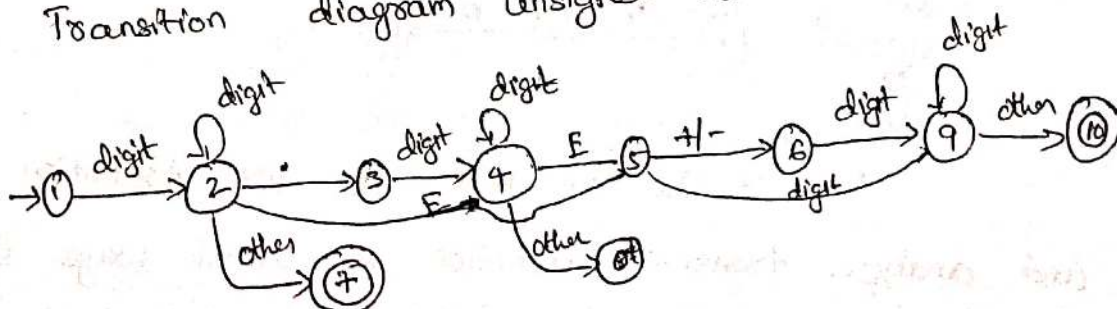
Transition diagram for relational operators



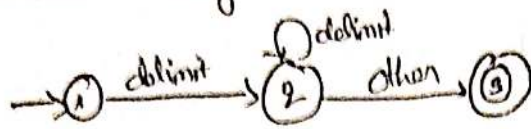
Transition diagram for identifier



Transition diagram unsigned numbers:



Transition diagram for white space:



Finite Automata :- Compiling a regular expression into a recognizer by constructing a generalized transition diagram is called finite automata. It is of two types ① DFA & ② NFA. Both are capable of recognizing the regular sets. The difference between NFA & DFA is in DFA for each input symbol it goes only to one state but NFA goes to several states.

eg: RE $(a/b)^*abb$ denotes set of all strings of a's & b's ending with abb.

Each Finite automata is defined by a set of 5- characters defined on $(Q, \Sigma, \delta, q_0, F)$ where

$Q \rightarrow$ set of states

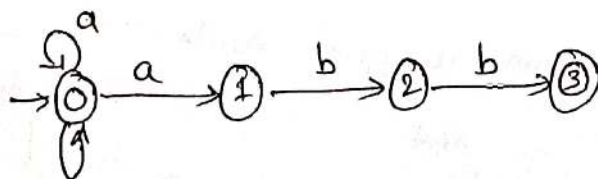
$\Sigma \rightarrow$ set of alphabets.

$\delta \rightarrow$ transition function from $Q \times \Sigma$

$q_0 \rightarrow$ initial state

$F \rightarrow$ set of final states

A NFA can be represented by labelled directed graph called transition graph, in which nodes are states and labelled edges represent the transition function.



To describe an NFA we use transition diagram and analyze transition function in several ways. But we use transition table for easiest implementation, in which

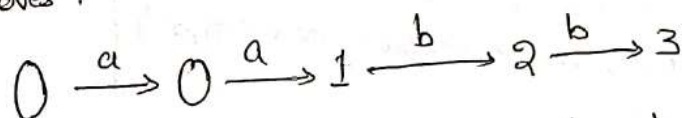
there is a row for each state & a column for each input symbol & ϵ if necessary. (10)

state	input symbols		ϵ
	a	b	
0	{0,1}	{0}	-
1	-	{2}	-
2	-	{3}	-
3	-	-	-

The main advantage of transition table is it provides fast access to the transitions of a given state on given character.

Its disadvantage is that it can take up a lot of space when the input is large & most transitions are empty.

A path can be represented by a sequence of state transitions called moves.



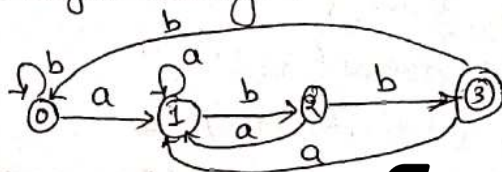
A NFA accepts input string x if and only if there is some path in the transition graph from the start state to one of the accepting state such that the symbols along the path spell out x .

Deterministic Finite Automata :-

A DFA is a special case of NFA in which

(i) No state has ϵ -transition

(ii) For each state S & input symbol, there is at most one edge leaving S .



Conversion of an NFA into DFA :-

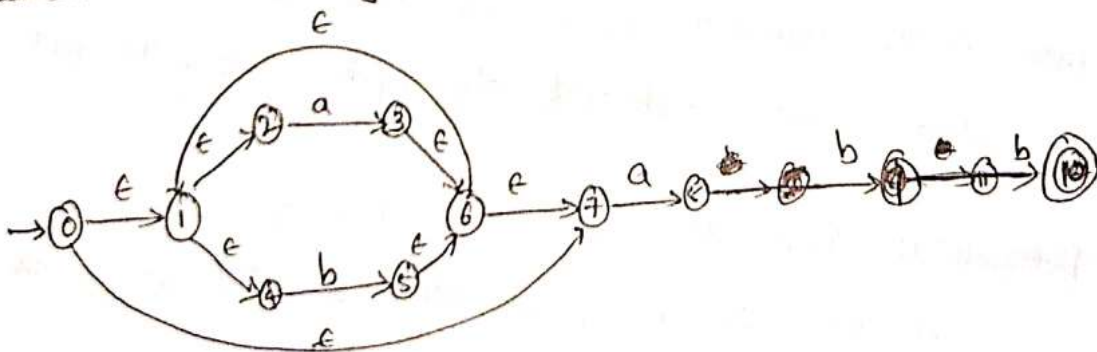
For converting an NFA into a DFA Depends on the following 3-questions :-

- (i) $\epsilon\text{-closure}(s) \rightarrow$ Set of NFA states reachable from NFA state s on ϵ -transition alone.
- (ii) $\epsilon\text{-closure}(T) \rightarrow$ Set of NFA states reachable from some NFA s in T on ϵ -transition alone.
- (iii) $\text{move}(T, a) \rightarrow$ Set of NFA states to which there is a transition input a from some NFA state s in T

First we consider the initial state by performing $\epsilon\text{-closure}(s)$ and perform $\text{DTrans}(A, a)$ where ' A ' is the initial state and ' a ' represents input alphabet. The ^{above} process is repeated until no new states are obtained.

$$\boxed{\text{DTrans}(A, a) = \epsilon\text{-closure}(\text{move}(A, a))}$$

Convert the following NFA with ϵ -moves to DFA.



sol:- $\epsilon\text{-closure}(0) \Rightarrow \{0, 1, 2, 4, 7\} - A$

$$\text{DTrans}(A, a) \Rightarrow \epsilon\text{-closure}(\text{move}(A, a))$$

$$\Rightarrow \epsilon\text{-closure}(\text{move}(\{0, 1, 2, 4, 7\}, a))$$

$$\Rightarrow \epsilon\text{-closure}(\{3, 8\})$$

$$\Rightarrow \{1, 2, 3, 4, 6, 7, 8\} - B$$

$$\text{DTrans}(A, b) \Rightarrow \epsilon\text{-closure}(\text{move}(\{0, 1, 2, 4, 7\}, b))$$

the

$$\Rightarrow \epsilon\text{-closure}(\{5\})$$

(11)

$$\Rightarrow \{1, 2, 4, 5, 6, 7\} = C$$

$$\text{Dtrans}(B, a) \Rightarrow \epsilon\text{-closure}(\text{move}(B, a))$$

$$\Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 3, 4, 6, 7, 8\}, a))$$

$$\Rightarrow \epsilon\text{-closure}(\{3, 8\})$$

$$\Rightarrow \{1, 2, 3, 4, 6, 7, 8\} = B$$

$$\text{Dtrans}(B, b) \Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 3, 4, 6, 7, 8\}, b))$$

$$\Rightarrow \epsilon\text{-closure}(\{5, 9\})$$

$$\Rightarrow \{1, 2, 3, 4, 5, 6, 7, 9\} = D$$

$$\text{Dtrans}(C, a) \Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 4, 5, 6, 7\}, a))$$

$$\Rightarrow \epsilon\text{-closure}(\{3, 8\})$$

$$\Rightarrow \{1, 2, 3, 4, 6, 7, 8\} = B$$

$$\text{Dtrans}(C, b) \Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 4, 5, 6, 7\}, b))$$

$$\Rightarrow \epsilon\text{-closure}(\{5\})$$

$$\Rightarrow \{1, 2, 4, 5, 6, 7\} = C$$

$$\text{Dtrans}(D, a) \Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 3, 4, 5, 6, 7, 9\}, a))$$

$$\Rightarrow \epsilon\text{-closure}(\{3, 8\})$$

$$\Rightarrow \{1, 2, 3, 4, 6, 7, 8\} = B$$

$$\text{Dtrans}(D, b) \Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 3, 4, 5, 6, 7, 9\}, b))$$

$$\Rightarrow \epsilon\text{-closure}(\{5, 10\})$$

$$\Rightarrow \{1, 2, 4, 5, 6, 7, 10\} = E$$

$$\text{Similarly Dtrans}(E, a) \Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 4, 5, 6, 7, 10\}, a))$$

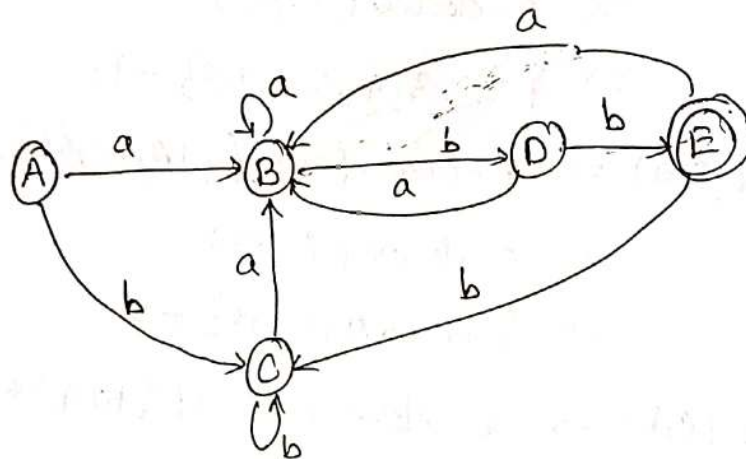
$$\Rightarrow B$$

$$\text{Dtrans}(E, b) \Rightarrow \epsilon\text{-closure}(\text{move}(\{1, 2, 4, 5, 6, 7, 10\}, b))$$

$$\Rightarrow C$$

The required transition table is as follows:-

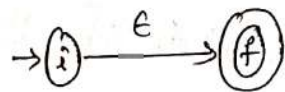
State	Input Symbols	
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C



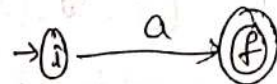
Construction of NFA from Regular Expression:-

For construction, it uses the following rules:

(i) For ϵ , Construct the NFA

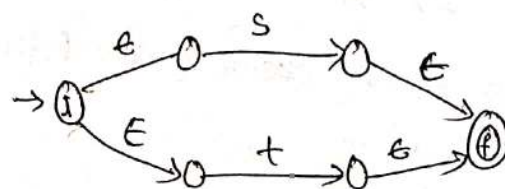


(ii) For a in Σ , Construct the NFA

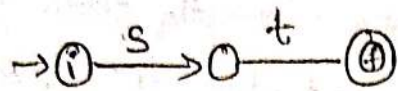


(iii) Suppose $N(s)$ & $N(t)$ are NFA's for R.E s & t

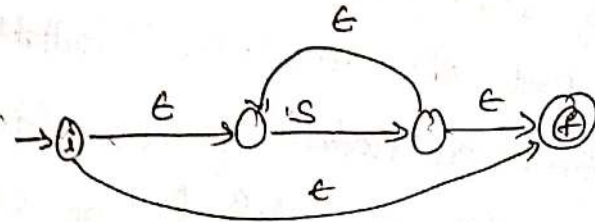
(a) For regular expression s/t Construct the NFA



(b) For regular expression st , Construct the NFA (12)



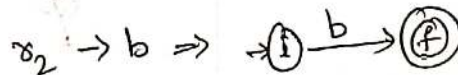
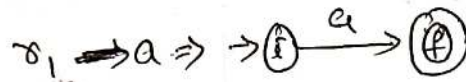
(c) For regular expression s^* , Construct the NFA



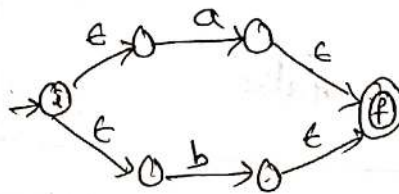
(d) For parenthesized RE (S) , use NCS) itself as NFA.

Construct NFA for regular expression $(a/b)^*abb$

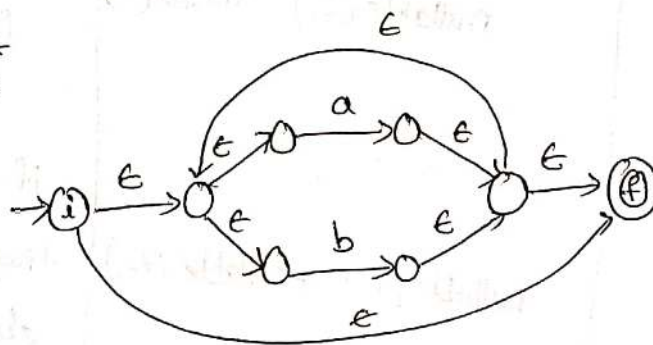
Sol:-



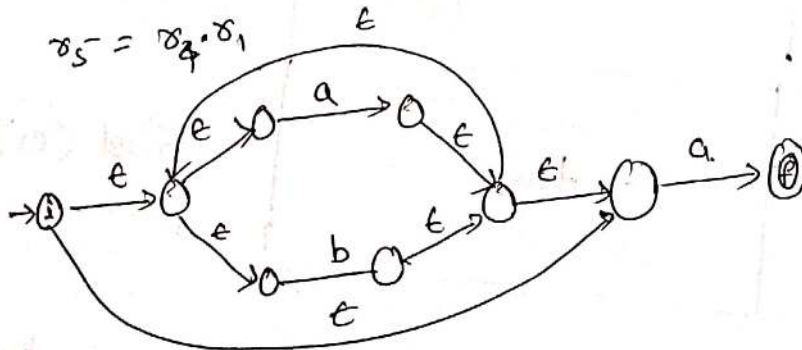
$r_3 = r_1 / r_2$



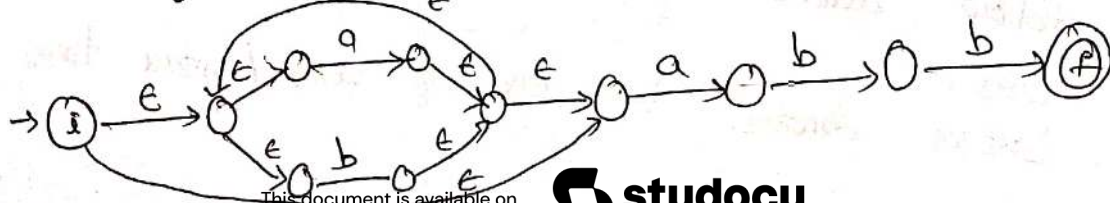
$r_4 = r_3^*$



$r_5 = r_4 \cdot r_1$



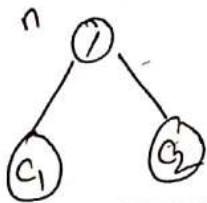
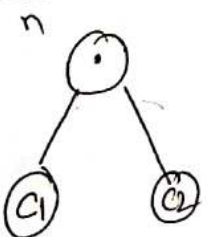
Similarly $r_6 = r_5 \cdot r_2$ & $r_7 = r_6 \cdot r_2$



Construction of DFA from Regular Expression:-

To construct a DFA directly from an augmented R.E $(R)\#$, we begin by constructing a syntax tree & compute following four functions; nullable, firstpos, lastpos & followpos. The first three functions nullable, first pos & lastpos are defined on nodes of a syntax tree and are used to compute followpos defined on set of position.

Rules for finding nullable & firstpos:-

Node N	nullable (n)	firstpos(n)
n is a leaf labelled with ϵ	True	ϕ
n is a leaf labelled with position i	False	$\{i\}$
	$\text{nullable}(c_1) \text{ or } \text{nullable}(c_2)$	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$
	$\text{nullable}(c_1) \& \text{nullable}(c_2)$	if $\text{nullable}(c_1)$ then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else $\text{firstpos}(c_1)$
	True	$\text{first}(c_1)$

follow denotes the next positions adjacent to the current ones.
Last pos denotes the end of parse/syntax tree.

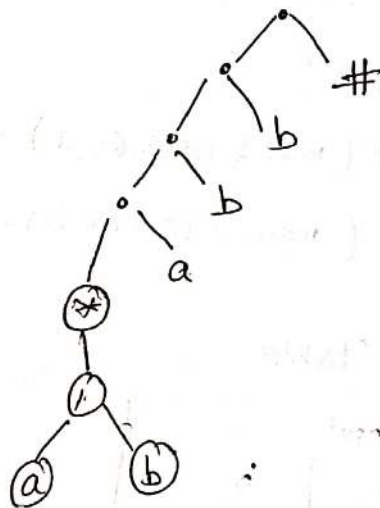
Method :

- ① Construct Syntax tree for the given regular expression.
- ② Construct the functions firstpos, lastpos, followpos & ^{nullable} ~~lastpos~~.
- ③ Construct Dstates, the set of states of D & Construct the transition table.

Construct DFA for regular expression $(a/b)^*abb$.

Sol:- $(a/b)^*abb\#$ is the augmented regular expression.

Syntax tree:-



nullable() = no {false}

firstpos() = {1, 2, 3} - A

last pos = {6}

followpos:-

i	follow(i)
1	{1, 2, 3}
2	{1, 2, 3}
3	{4}
4	{5}
5	{6}
6	

$$DTrans(A, a) \Rightarrow follow(move(first(1), a)) \Rightarrow follow(move(\{1, 2, 3\}, a)) \Rightarrow \\ \Rightarrow follow(\{1, 3\}) \Rightarrow \{1, 2, 3, 4\} - B$$

$$DTrans(A, b) \Rightarrow follow(move(first(1), b)) \Rightarrow follow(move(\{1, 2, 3\}, b)) \\ \Rightarrow follow(\{2, 3\}) \Rightarrow \{1, 2, 3\} - A$$

$$DTrans(B, a) \Rightarrow follow(move(\{1, 2, 3, 4\}, a)) \Rightarrow follow(\{1, 3\}) - B$$

$$DTrans(B, b) \Rightarrow follow(move(\{1, 2, 3, 4\}, b)) \Rightarrow follow(\{2, 5\}) - \\ \Rightarrow \{1, 2, 3, 5\} - C$$

$$DTrans(C, a) \Rightarrow follow(move(\{1, 2, 3, 5\}, a)) \Rightarrow follow(\{1, 3\}) - B$$

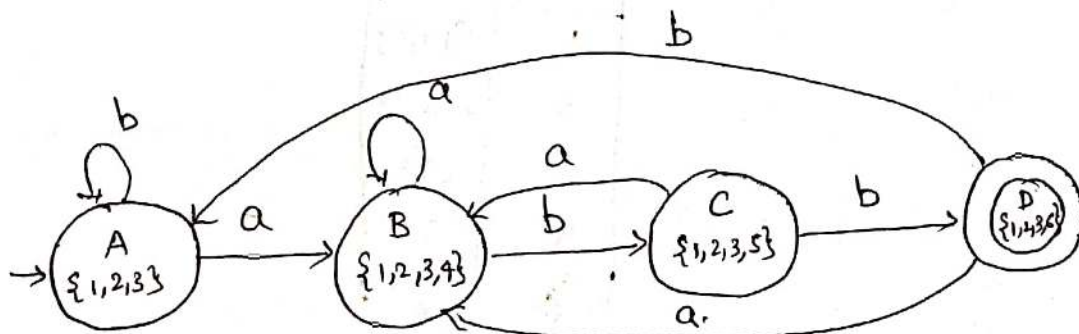
$$DTrans(C, b) \Rightarrow follow(move(\{1, 2, 3, 5\}, b)) \Rightarrow follow(\{2, 6\}) \\ \Rightarrow \{1, 2, 3, 6\} - D$$

$$DTrans(D, a) \Rightarrow follow(move(\{1, 2, 3, 6\}, a)) \Rightarrow follow(\{2, 3\}) - B$$

$$DTrans(D, b) \Rightarrow follow(move(\{1, 2, 3, 6\}, b)) \Rightarrow follow(\{2, 3\}) - A$$

Transition Table

	Input	
	a	b
→ A	B	A
B	B	C
C	B	D
ⓓ	B	A



Minimization of the number of states in a DFA

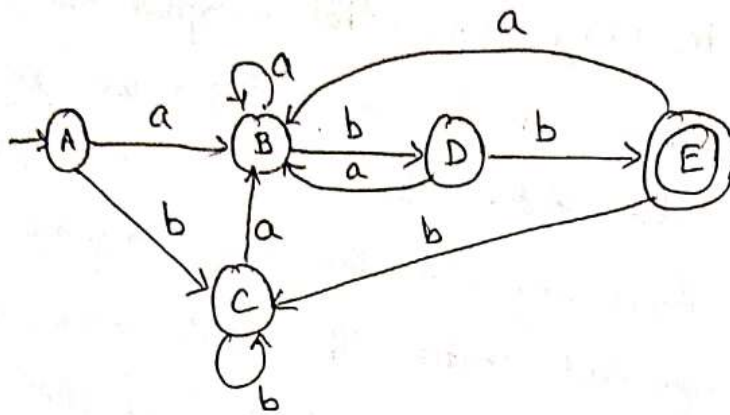
(14)

There can be many DFA's that recognize the same language. If we implement a lexical analyzer we generally prefer a DFA with few states. The minimum-state DFA can be constructed from any DFA for some language by grouping sets of equivalent states. The state-minimization algorithm works by partitioning the states of a DFA into groups of states that cannot be distinguished.

Method:

1. Start with initial partition π with two groups, F & $S-F$, the accepting & non-accepting states of D .
2. Apply the procedure to construct a new partition π_{new} initially $\pi_{\text{new}} = \pi$, for (each group G of π)
partition G into subgroups such that two states $s \neq t$ are in same subgroup if & only if for all input symbols a , state $s \neq t$ have transitions on a to states in the same group of π .
replace G in π_{new} by set of all subgroups formed.
3. If $\pi_{\text{new}} = \pi$ let $\pi_{\text{final}} = \pi$ & continue with step (4) otherwise ~~goto~~ repeat step (2) with π_{new} in place of π .
4. choose one state in each group of π_{final} as the representative for that group. The representatives will be the states of minimum state DFA D' .

Construct minimized DFA for the following DFA:-



sol:- (i) The initial partition consists of two groups $\{A, B, C, D\}$ $\{E\}$ which are non-accepting & accepting states.

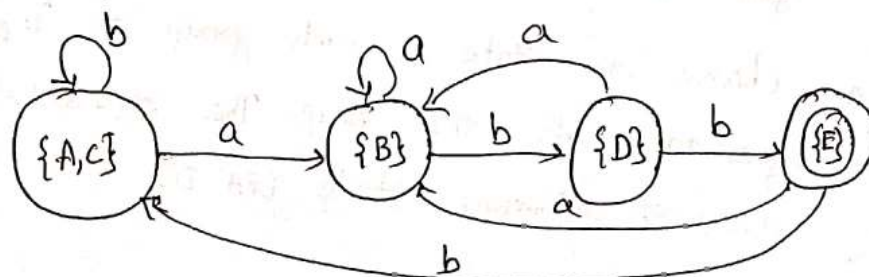
(ii) The group $\{E\}$ cannot be partitioned.

(iii) The other group $\{A, B, C, D\}$ can be partitioned into two groups $\{A, B, C\}$ $\{D\}$. On each input a , each state goes to B but on input b , states $\{A, B, C\}$ goes to group $\{A, B, C, D\}$ while D goes to group $\{E\}$.

(iv) The group $\{D\}$ cannot be partitioned.

(v) The other group $\{A, B, C\}$ can be partitioned into two groups $\{A, C\}$ $\{B\}$. On each input a , each state goes to the group $\{A, B, C\}$ but on input b , states $\{A, C\}$ goes to group $\{A, B, C\}$ while state $\{B\}$ goes to group $\{D\}$.

(vi) The states that are formed are $\{A, C\}$, $\{B\}$, $\{D\}$, $\{E\}$.



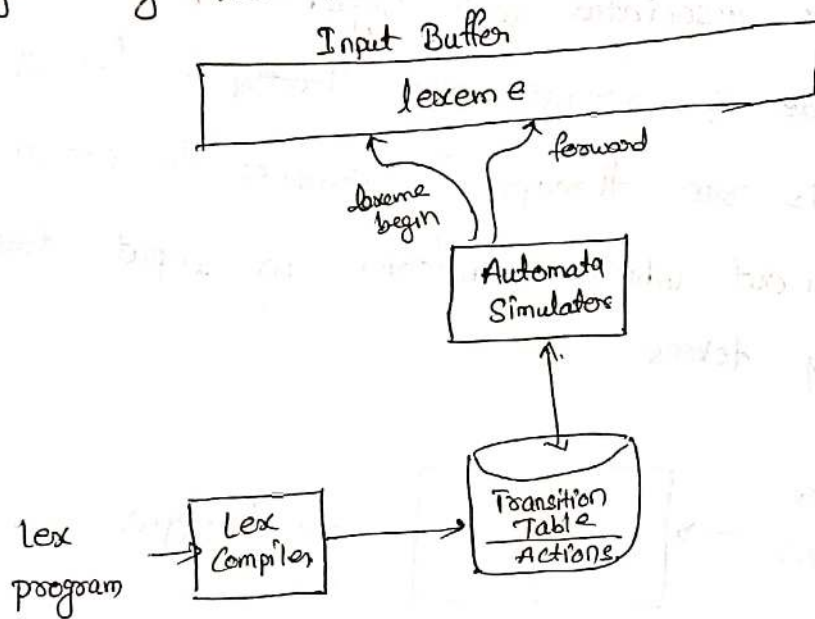
Minimized DFA

(iii) Structure

Design of a Lexical-Analyzer Generator :-

(16)

The architecture of a lexical analyzer generated by lex includes a fixed program that simulates an automaton which is deterministic or non-deterministic. The rest of lexical analyzer consists of components that are created from lex program by Lex itself.



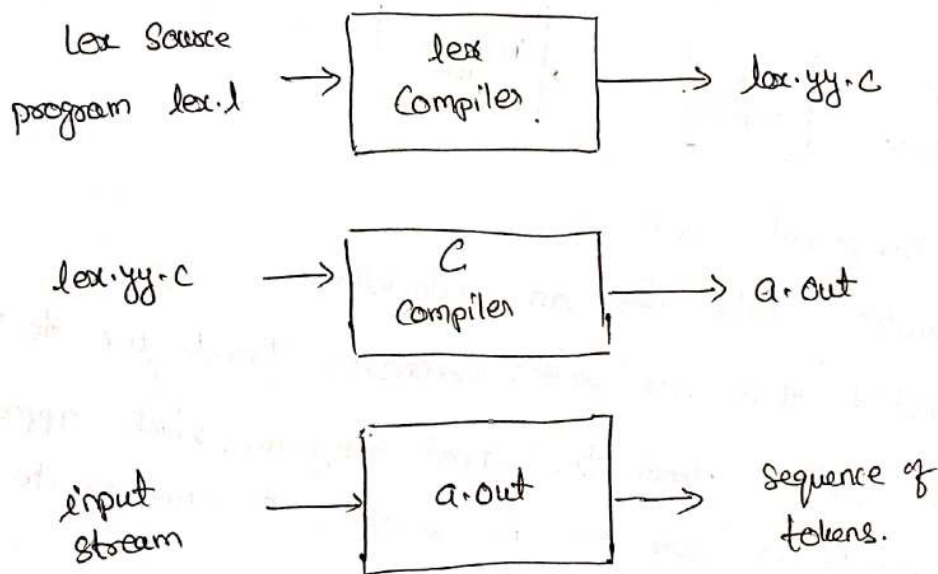
The components are

- (i) Transition table for an automaton
- (ii) Function that are passed directly through Lex to the output.
- (iii) The actions from the input program, which appear as fragments of code to be invoked at appropriate time by the automaton simulator.

LEX TOOL / Language for specifying Lexical Analyzer :-

The tool is used to specify lexical analyzers for a wide variety of languages is Lex Compiler. This tool allow us to show the specification of patterns using R-E (regular expression) combined with actions.

A lexical analyzer is first prepared by creating a program `lex.l` in Lex language. This program `lex.l` is run through lex Compiler to produce a program `lex.yy.c`. This program `lex.yy.c` consists of a transition diagram constructed from regular expression of `lex.l`, to recognize tokens/lexeme. The actions associated with regular expressions are pieces of C code & carried over directly to `lex.yy.c`. Finally, `lex.yy.c` is run through C compiler to produce an object program `a.out` which transforms an input stream into sequence of tokens.



A lex program consists of 3 parts:

- (i) Declarations part: The declaration part includes variables, constants & regular expressions. It begins with `%{` & ends with `%}`
- (ii) Translation rules part: The translation rule of a lex program are statements of the form

P₁ { action₁ }

P₂ { action₂ }

⋮

P_n { action_n }

where P_i is a regular expression

action_i describe what action LA take to match lexeme.

(iii) Auxiliary Procedures part :- The third section holds what auxiliary procedures need for action.. These procedures are Compiled separately & loaded into Lexical Analyser.

/* {

// declarations

*/ }

/* /*

translation rules

/* /*

auxiliary Procedures.

Lex Program to generate tokens :-

/* {

/* definition of constants

LE, LTE, GE, GTE, EQ, NE

IF, THEN, ELSE, ID, NUMBER */

*/ }

/* regular Definitions */

delim [\t\n]

ws { delim⁺ }

letter [A-Za-z]

digit [0-9]

id { letter } ({ letter } / { digit })^{*}

number { digit }⁺ ((. { digit }⁺) ? (E(+|-) ? { digit }⁺) ?)

%. % ,

{ ws } { // no action & no return #/ }

if { return (IF); }

then { return (THEN); }

else { return (ELSE); }

{ id } { yyval = (int) install-ID(); return (ID); }

{ number } { yyval = (int) install-Num(); return (NUMBER); }

"<" { yyval = LT; return (RELOP); }

">" { yyval = GT; return (RELOP); }

"<=" { yyval = LTE; return (RELOP); }

">=" { yyval = GTE; return (RELOP); }

"<>" { yyval = NAE; return (RELOP); }

"==" { yyval = EQ; return (RELOP); }

%. % ,

int install-ID()

{

}

int install-Num()

{

}

(17)

Following are the lex predefined functions and variables :-

- yyin** :- the input stream pointer (i.e it points to an input file which is to be scanned or tokenised), however the default input of default main() is stdin .
- yylex()** :- implies the main entry point for lex, reads the input stream generates tokens, returns zero at the end of input stream . It is called to invoke the lexer (or scanner) and each time yylex() is called, the scanner continues processing the input from where it last left off.
- yytext** :- a buffer that holds the input characters that actually match the pattern (i.e lexeme) or say a pointer to the matched string .
- yytext** :- the length of the lexeme .
- yyval** :- contains the token value .
- yyval** :- a local variable .*
- yyout** :- the output stream pointer (i.e it points to a file where it has to keep the output), however the default output of default main() is stdout .
- yywrap()** :- it is called by lex when input is exhausted (or at EOF). default yywrap always return 1.
- yywrap()** :- returns the next token .
- yyless(k)** :- returns the first k characters in yytext .
- yyparse()** :- it parses (i.e builds the parse tree) of lexeme .*

Conflict Resolution in Lex :-

The two rules that Lex uses to decide on proper lexeme to select when several prefixes of the input matches one or more patterns :

- (i) Always prefer a longer prefix to a shorter prefix.
- (ii) If the longest possible prefix matches two or more patterns prefer the pattern listed first in the lex program.